Orchestrate agents and workstreams

Product and delivery teams, or developers after Session M2-05

Session M7-05 | 3 hours

Agenda

Segment	Time
Parent objective, terms, and boundaries	8 min
When to split, independence, shared dependencies	10 min
Minimal workstream plan	8 min
Monitor and intervene; failure modes	12 min
Live demonstration	12 min
Review and lab handoff	10 min

Three terms

- **Parent-led orchestration:** one parent session owns the objective, starts children, and reviews what they return.
- **Child session:** a separate session that works one question from its own inputs.
- **Stop condition:** the one boundary that ends a child's work.

The parent keeps decision authority.

Orchestration, parallel chats, delegation

Approach	Difference
Parallel chats	You merge answers by hand. No parent reviews.
Delegation (M2-05, M3-02)	One task to one agent. You review one output.
Orchestration	A parent splits one objective, runs children, reconciles them.

When to split

Split when:

- the questions are independent;
- each child has its own inputs;
- the parent can check each result.

Keep it in one session when each step needs the last answer, or the work is small enough that splitting costs more than it saves.

Make workstreams independent

- One question per child, from separate inputs.
- Children don't read each other's results.
- **Shared dependency?** Move it to the parent, or settle it before you start.
- Overlap between two children means the split is wrong. Redraw it.

Required access

- `/orchestrate` creates and coordinates live child sessions (check current docs for your app version).
- Product route: GitHub MCP reads the selected issues.
- Developer route: local project read access and Python 3.10 or later.
- You can open, redirect, and stop child sessions.

Choose one lab route

Route	Inputs	Result
Product and delivery	Four live GitHub issues	Parent result used by Session M7-06
Developer	Your M2-05 fix, tests, and pull request	Separate implementation, coverage, and PR conclusions

Each route takes 120 minutes. Developers need Session M2-05, not Sessions M7-01 to M7-04.

One parent objective

Review the four approved status-visibility issues and return one reviewed result for each.

The parent owns scope and review.

Keep the plan small

Each workstream needs:

- a question with a clear scope;
- approved inputs;
- a stop condition;
- a reviewer.

The orchestration surface already holds session links, status, timing, and usage.

Four independent workstreams

Workstream	Question
Public mapping	Is the mapping ready for approval?
Requester status	Is the issue ready for implementation review?
Empty and stale states	Are fallback states testable?
Access boundary	Are the approved requester-access results ready?

Move shared questions back to the parent.

Developer workstreams

Question	Input
Does the fix meet `issue-brief.md`?	Brief and `shop/pricing.py`
What do tests prove or miss?	Brief, tests, and `shop/pricing.py`
What do the PR's checks and threads say?	The real PR, read with `gh` or GitHub MCP

Children don't wait for one another, and they don't edit.

Practise steering

Redirect the test reviewer after its first answer:

```
Map each rule in issue-brief.md to the test that would catch it. Which rules
have no test? Do not edit files.
```

Inspect the revised answer. A follow-up doesn't prove the first answer was wrong.

Reconcile the readiness claim

The author note says: "Local tests pass, so the PR is ready."

Check the real PR:

- is the `tests` check green on the head commit?
- are any review threads unresolved?

Local success doesn't override a red check or an open thread.

Start live orchestration

```
/orchestrate Use the four workstreams in my approved plan. Keep them independent.
Each result must answer its question, cite approved inputs, state unresolved
gaps, and give a recommendation. Obey each stop condition.
```

Monitor on the platform

Signal	Action
Safe and on scope	Wait
Safe work needs correction	Redirect
A boundary is crossed	Stop
The result meets the plan	Accept
The result lacks support	Reject
Human authority is required	Escalate

Review each result

- Does it answer the assigned question?
- Does it use only the approved issues and their linked sources?
- Did it obey the stop condition?
- Are gaps listed?
- Is the recommendation supported?

Reject answers that lack supporting sources.

Failure modes

Failure	Response
Hidden dependency: a child needs another's answer	Pause one; move the question to the parent
Overlap: two children answer the same thing	Merge or redraw
Child drifts past its question	Redirect
Child crosses the stop condition	Stop
Confident answer, no source	Reject
Children agree because they share a source	Check the source, not the count

The parent result is the handoff

The reviewed result:

- links findings to live GitHub issues;
- shows rejected claims;
- names gaps and owners;
- recommends the next follow-up.

Session M7-06 reads this result and the live issues.

The human keeps authority

Copilot coordinates work. A person reviews results and approves any issue update, publication, or release statement.

Session M5-01 is different

Session M5-01	Session M7-05
Persistent roles and routing	Temporary workstreams
Optional framework setup	Built-in orchestration
Shared memory	Live issues and their supporting sources

Lab deliverable

1. Approved orchestration plan
2. Live child-session results
3. Reviewed parent result linked to live issues

Keep progress on the orchestration surface, not in a status form.

For developers, use the developer plan and reviewed parent result instead of live issues. Include the actual redirection, rejected or unresolved claims, and observed usage.